

**Autosoft**

Le SaaS qui libère les auto-écoles françaises

DOCUMENTATION TECHNIQUE

Architecture Autosoft : convention canonique

Convention canonique d'organisation du code

Document confidentiel

Diffusion restreinte : Autosoft & partenaires

21 avril 2026

auto-soft.fr

Sommaire

1. Module de référence

2. Arborescence d'un module

3. Sous-features (components/[FeatureName]/)

Sous-feature vs sous-route : ne pas confondre

Convention de nommage

Structure interne d'une sous-feature

Règles strictes

4. Conventions de nommage

5. Routes API

Où placer une route ?

Squelette obligatoire

Règles strictes

6. Composants partagés (src/components/)

Règles strictes

7. Lib métier (src/lib/)

8. Hooks et contextes globaux

9. Tests

10. Interdictions explicites

11. État actuel : zones conformes et zones à migrer

Conformes (modèles)

À migrer (PRs futures, non bloquant)

12. Quand cette convention évolue

Source de vérité pour l'organisation du code côté Next.js (`autosoft/src/`). Toute nouvelle feature DOIT respecter cette convention. Les zones encore divergentes sont listées en fin de document.

Voir aussi : `AI_CONTEXT.md` pour le contexte produit/business, et `ARCHITECTURE_ANALYSIS.md` pour l'analyse historique qui a abouti à ces règles.

1. Module de référence

Le dossier `autosoft/src/app/(protected)/directeur/eleves/` est le **modèle canonique**. Tout nouveau module ou feature doit s'en inspirer directement.

2. Arborescence d'un module

Un "module" = un espace fonctionnel sous un rôle (ex : `directeur/eleves`, `directeur/paiements`, `moniteur/lecon`).

```
module/
├── page.tsx                # Page principale (orchestration uniquement)
├── layout.tsx              # Layout du module si spécifique
├── api/                    # APIs globales du module
│   └── [resource]/route.ts
├── components/             # Composants du module
│   ├── shared/            # Helpers partagés entre features (utils.ts, hooks)
│   ├── index.ts           # Re-exports publics du module
│   ├── [Composant].tsx     # Composants simples (Header, Modal, etc.)
│   └── [FeatureName]/      # Sous-feature (cf. §3)
├── hooks/                  # Hooks transverses au module
├── types/                  # Types partagés du module (ou types.ts à la racine)
├── new/                    # Sous-route "création" (kebab-case)
│   ├── page.tsx
│   ├── api/
│   ├── components/
│   ├── hooks/
│   └── types/
└── [id]/                   # Sous-route détail dynamique
    ├── page.tsx
    ├── api/
    └── components/        # Onglets ou sections du détail
        └── [tabName]/    # Un onglet = une sous-feature complète
```

3. Sous-features (`components/[FeatureName]/`)

Une **sous-feature** est un bloc fonctionnel autonome à l'intérieur d'un module (ex : `ContratsFeature`, `EcheancesFeature`, `CreneauAjoutFeature`). Elle vit **dans** `components/`, n'a **pas de** `page.tsx` propre, et est consommée par la `page.tsx` du module via un import.

Sous-feature vs sous-route : ne pas confondre

	Sous-route (App Router)	Sous-feature (composant)
Localisation	À la racine du module	Dans <code>module/components/</code>
Fichier d'entrée	<code>page.tsx</code>	<code>index.tsx</code>
URL dédiée	Oui (<code>/module/sous-route</code>)	Non
Nommage	kebab-case (<code>new/</code> , <code>examen-blanc/</code> , <code>mon-auto-ecole/</code>)	PascalCase + Feature (<code>ContratsFeature/</code>)
Exemple	<code>moniteur/lecon/competences/page.tsx</code>	<code>directeur/paiements/components/ContratsFeature/index.tsx</code>

Un module peut avoir les deux : sous-routes pour les écrans aux URLs distinctes (wizards multi-étapes, formulaires de création), sous-features pour les blocs intégrés à une page unique (onglets, sections).

Convention de nommage

- **Sous-feature (dans `components/`)** : PascalCase **avec suffixe Feature** : `ContratsFeature` , `PrestationsFeature` , `CreneauAjoutFeature` .
- **Sous-features pour onglets de détail** sous `[id]/components/` : kebab-case (calqué sur l'URL conceptuelle), avec `index.tsx` PascalCase à l'intérieur. Exemples acceptés : `[id]/components/informations/` , `[id]/components/planning/` .
- **Composants simples** (non autonomes, pas de sous-fichiers, pas d'API propre) : un seul fichier `Composant.tsx` à la racine de `components/` , pas de dossier.

Structure interne d'une sous-feature

```
[FeatureName]/
├── index.tsx           # OBLIGATOIRE : point d'entrée exporté
├── api/
│   └── route.ts       # Si APIs spécifiques à la feature
├── components/
│   ├── [SubComponent].tsx
│   └── [DeepFeature]/  # Sous-feature imbriquée si justifié
├── hooks/
│   └── use[Name].ts    # Logique métier extraite
├── context/           # Optionnel : Context API si état partagé profond
│   └── [Name]Context.tsx
├── utils/             # Optionnel : fonctions pures
│   └── [name]Utils.ts
└── types.ts           # Types locaux à la feature
```

Règles strictes

1. `index.tsx` est **obligatoire** comme point d'entrée d'une sous-feature.
2. **Colocation** : tout ce qui concerne la feature (API, hooks, types, composants) reste à l'intérieur du dossier.
3. **Pas de logique métier dans `page.tsx`** : la page orchestre (état d'onglet actif, mise en page), les features font le travail.
4. **Logique métier dans des hooks** (`useContrats` , `useFiltresPlanning`), jamais inline dans le JSX.
5. **Types locaux** : `types.ts` dans la feature. Promouvoir vers `module/types/` uniquement si réellement partagé entre plusieurs features.

4. Conventions de nommage

Élément	Convention	Exemple
Dossier de module	kebab-case (route Next.js)	<code>mon-auto-ecole/</code> , <code>paiements/</code>
Sous-route Next.js	kebab-case	<code>new/</code> , <code>[id]/</code> , <code>demandes-dossier/</code>
Sous-feature (composant React)	PascalCase + <code>Feature</code>	<code>ContratsFeature/</code> , <code>EcheancesFeature/</code>
Onglet de détail (<code>[id]/components/x/</code>)	kebab-case	<code>informations/</code> , <code>planning/</code> , <code>ants/</code>
Composant React (fichier)	PascalCase	<code>EleveTable.tsx</code> , <code>CreneauModal.tsx</code>
Hook	camelCase préfixé <code>use</code>	<code>useContrats.ts</code> , <code>useFiltresPlanning.ts</code>
API client (helper)	camelCase suffixé <code>Api</code>	<code>eleveApi.ts</code> , <code>contratDetailApi.ts</code>
Utilitaire	camelCase	<code>validationUtils.ts</code> , <code>formatPrix.ts</code>
Type / Interface	PascalCase	<code>EleveFormData</code> , <code>ContratFormValue</code>
Constantes globales	SCREAMING_SNAKE_CASE	<code>TYPES_MANOEUVRES</code>

5. Routes API

Où placer une route ?

Portée	Emplacement	Exemple
Globale, partagée entre modules	<code>src/app/api/...</code>	<code>src/app/api/utils/forfaits/route.ts</code>
Spécifique à un module	<code>module/api/...</code>	<code>directeur/eleves/api/ElevesTable/route.ts</code>
Spécifique à une sous-feature	<code>module/components/[Feature]/api/...</code>	<code>paiements/components/ContratsFeature/api/route.ts</code>
Spécifique à une page de détail	<code>module/[id]/api/...</code>	<code>paiements/contrats/[id]/api/generate-pdf/route.ts</code>

Squelette obligatoire

Toute route API protégée doit suivre ce pattern (cf. `SECURITE_API.md`) :

```
import { NextRequest, NextResponse } from 'next/server';
import { createClientWithRLS } from '@lib/supabase-rls';
import { verifyAuth } from '@lib/security/auth-helpers';

export async function GET(request: NextRequest) {
  try {
    const authResult = await verifyAuth(request);
    if (!authResult.success || !authResult.user) {
      return NextResponse.json({ error: 'Non authentifié' }, { status: 401 });
    }

    const supabase = await createClientWithRLS();
    const id_ecole = authResult.user.id_ecole; // Toujours depuis le JWT

    const { data, error } = await supabase
      .from('table')
      .select('*')
      .eq('id_ecole', id_ecole);

    if (error) throw error;
    return NextResponse.json(data);
  } catch (error) {
    console.error('[Module] Erreur GET:', error);
    return NextResponse.json({ error: 'Erreur serveur' }, { status: 500 });
  }
}
```

Règles strictes

- 1. `id_ecole` et `id_moniteur` : toujours depuis `authResult.user`, jamais depuis le body ou les query params.
- 2. `createClientWithRLS()` par défaut. `supabaseAdmin` uniquement avec justification documentée.
- 3. Pas de PII dans les `console.error` (utiliser des IDs).

6. Composants partagés (`src/components/`)

Dossier	Rôle
<code>src/components/ui/</code>	Source unique des primitives (Card, Button, Input, Select, Dialog, etc.). Basée sur shadcn/ui + Radix.
<code>src/components/layout/</code>	Layouts par rôle : <code>DirectorLayout</code> , <code>MoniteurLayout</code> , <code>EleveLayout</code> .
<code>src/components/shared/</code>	Composants métier réutilisables entre modules (ex : <code>OfflineOverlay</code>).
<code>src/components/contrats/</code>	Composants de domaine partagés entre plusieurs modules (ex : <code>ContratForm</code> utilisé par directeur/paiements/contrats/new ET le wizard d'inscription élève).
<code>src/components/onboarding/</code> , <code>marketing/</code> , <code>notifications/</code> , <code>selectors/</code> , <code>ChatWidget/</code>	Domaines fonctionnels transverses.

Règles strictes

- 1. **Une seule source pour les primitives UI** : `src/components/ui/`. Interdiction de créer un dossier `ui/` à l'intérieur d'un module.
- 2. Si un composant primitif manque (ex : Button avec variante "success"), **étendre** `src/components/ui/button.tsx` plutôt que créer un wrapper local.
- 3. Un composant n'est promu vers `src/components/` qu'à partir du moment où il est utilisé par **au moins deux modules différents**. Sinon il reste local.

7. Lib métier (`src/lib/`)

Dossier	Rôle
<code>src/lib/security/</code>	Auth, JWT, RBAC, helpers de session.
<code>src/lib/ants/</code>	Client ANTS (proxy VPS, mapping CERFA, credentials).
<code>src/lib/contrats/</code>	Helpers métier contrats (<code>buildPayload</code> , <code>calculatePrix</code>).
<code>src/lib/supabase*.ts</code>	Clients Supabase (browser, server, RLS, admin).

Pattern : un dossier par domaine métier, fonctions pures testables, pas de JSX.

8. Hooks et contexts globaux

- Hook spécifique à une feature : `feature/hooks/useX.ts` .
- Hook utilisé par plusieurs features d'un même module : `module/hooks/useX.ts` .
- Hook utilisé par plusieurs modules : `src/hooks/useX.ts` .
- Context global (utilisateur connecté, thème) : `src/contexts/` .

9. Tests

- Runner : Vitest.
- Tests unitaires (logique pure) : `tests/unit/<domaine>/<nom>.test.ts` .
- Tests d'intégration API : `tests/integration/api/<route>.test.ts` .
- Setup global : `tests/setup.ts` (mocks Supabase + env vars factices).
- Toute nouvelle route API protégée doit avoir au minimum un test "401 sans auth" et un test d'isolation `id_ecole` .

10. Interdictions explicites

- Pas de logique métier dans `page.tsx`** : orchestration uniquement.
- Pas de wrapper local autour des primitives shadcn** : étendre la primitive elle-même.
- Pas de fichier `> 600 lignes`** : extraire en hooks, sous-composants, helpers.
- Pas de `any`** sauf cas justifié et commenté.
- Pas de filtre `id_ecole` venant du client** : toujours du JWT.
- Pas de doublon de structure** : si une feature existe dans un module, ne pas la dupliquer dans un autre, la remonter dans `src/components/<domaine>/` .
- Pas de mélange de conventions de nommage** dans un même dossier.

11. État actuel : zones conformes et zones à migrer

Conformes (modèles)

- `directeur/elevés/` : référence absolue.
- `directeur/paiements/components/{ContratsFeature, PaiementsFeature, ForfaitsFeature, PrestationsFeature, EcheancesFeature, AcomptesFeature, StatistiquesFeature}/` : structure feature-based correcte.

- `directeur/planning/components/{CreneauAjoutFeature, PlanningFiltersFeature, PlanningGridFeature}/` : conforme.
- `directeur/comptabilite/components/{ComparaisonFeature, DepensesFeature, RecettesFeature, ResultatFeature, TVAFeature}/` : aligné depuis le commit `62cc74c` (ex Comparaison/Depenses/Recettes/Resultat/TVA).
- `moniteur/lecon/{competences, observation, manoeuvre, questions, examen-blanc, terminer}/` : conforme. Ces dossiers sont des **sous-routes** App Router (chacun a un `page.tsx` et une URL propre `/moniteur/lecon/<etape>`), pas des sous-features. Le kebab-case est donc la convention attendue (cf. §3 et §4).
- `directeur/paiements/components/` : nettoyé. Le dossier `shared/ui/` (wrappers redondants autour de shadcn) et le re-export mort `components/index.ts` ont été supprimés. Les 5 consommateurs (`ContratsTable` , `PaiementsTable` , `PrestationsTable` , `ForfaitsTable` , `StatistiquesCards`) importent désormais directement depuis `@/components/ui/{card,badge,button}` .

À migrer (PRs futures, non bloquant)

Aucune zone connue à ce jour. Cette section sera mise à jour si une nouvelle divergence est identifiée.

12. Quand cette convention évolue

Toute modification de cette convention doit :

1. Être discutée explicitement (pas un changement implicite via un PR de feature).
2. Mettre à jour ce fichier ET l'INDEX de documentation.
3. Si elle invalide du code existant, lister les zones à migrer dans la section §11.